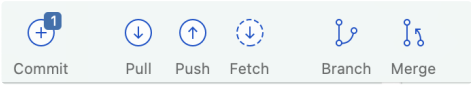
Git Basics

Git is a version control tool used by almost all software development teams for managing changes made across a large team. Git has a steep learning curve and while powerful, also allows users to accidentally make fairly major mistakes, so it's important everyone understand the fundamentals and ask for help if they encounter an unfamiliar issue.

> We recommend making regular backups of your work and iterating on small frequent commits to avoid losing a large amount of work.

Every person has their own preferred workflow and use their own preferred git client so the following information may vary for you.

How Does Git Work?



SourceTree buttons for common actions

Git will manage your local copy of the project (**Repository**) and a remote copy shared by everyone.

A **Commit** can be thought of as a packet of changes you want applied.

Pushing your commits will sync the changes to the remote repository so everyone can access them.

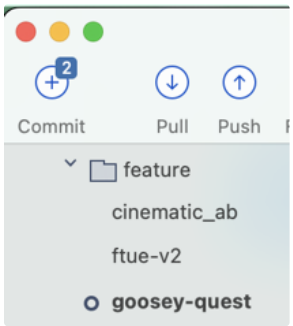
Pulling latest will sync your local repository with the remote.

Branches are Git's way of managing commits in isolation before **Merging** them back into the main branch. Branching is useful as a playground for testing out ideas, working on a longer term feature, or letting you switch easily between several tasks without cluttering up your Stage.

The **Stage** is the set of changes ready to be committed. You can have other file changes "waiting in the wings" but anything staged will get committed so always double check your work!

Simplest Workflow

1. Pull the latest changes on the branch you're working on (development, a release branch, or a feature branch)



SourceTree will usually tell you if there are changes to pull but it's always safe to click Pull just in case it hasn't refreshed

2. Do your work locally. Test your changes in game!
3. Check your git client (eg SourceTree, Fork, Github Desktop, etc) to see the staged changes

You can preview the diff in most git clients to see what you changed

4. Verify all the changes look correct and related to what you did (file names are a great hint)
5. Pull again. It's easier to avoid merge conflicts if you're up to date before a commit is made.
   a. If you get an error here it means one of your files will probably conflict. Discard the relevant file, pull, and then redo your change.
6. Commit your changes. Make sure to include a descriptive commit message

Optionally you can check the "Push immediately" box but I prefer to commit and push as two steps

6. Push your changes

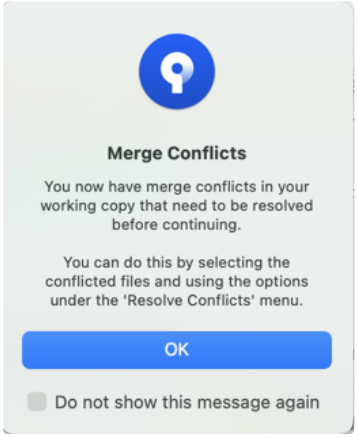
Merge Requests

Sometimes you'll want to have your work double checked by someone else before merging it into the main development branch. Merge Requests are a feature of GitLab that let you request a review from an engineer.

1. Pull latest of the branch you're trying to add to
2. Create a branch off that branch. Name it something like *dev/your_name/descriptive_branch_name* (keep it lowercase and spaceless for simplicity)
3. Do your changes as normal. Commit and push your changes to your custom branch
4. Once ready for review go to gitlab.com , navigate to Merge Requests, and click New Merge Request
   a. https://gitlab.com/squid-games/starfish/-/merge_requests
5. Check the MR for issues like merge conflicts.
   a. If you have a merge conflict, go back to your branch and pull the target branch. Do **not** rebase! Resolve the conflict as usual and push.
6. Fill in some info on what your changes are doing. Add Jeff or Caroline as a reviewer
7. If there are no issues, we'll merge your branch in. Otherwise check your email or the MR for comments. You may need to make some changes on your branch and commit/push those changes. The MR will update automatically with the new changes.

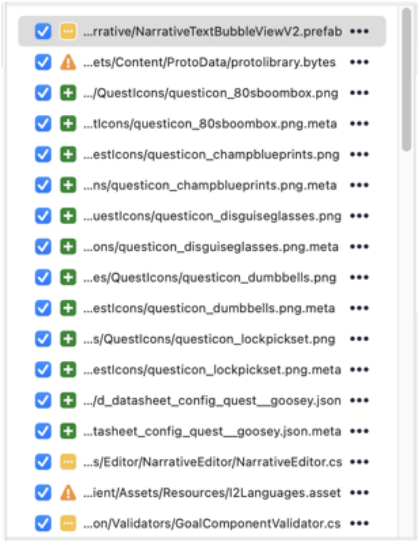
Merge Conflicts

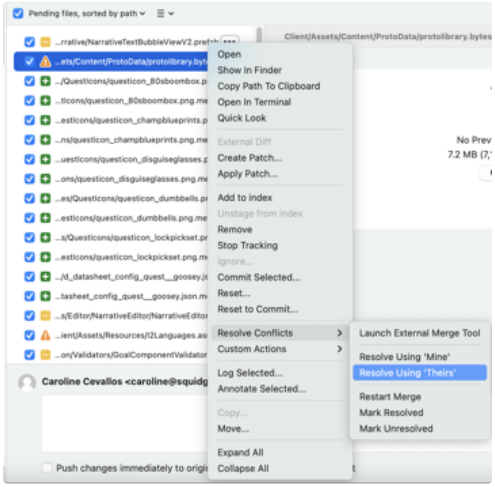
Oftentimes when you pull latest and have local commits not yet pushed you'll run into what is known as a merge conflict. This is when a file change on the remote repository conflicts with the local changes you've made, and Git doesn't know how to resolve them. You **must** resolve merge conflicts before doing anything else! This is where your workflow is going to change based on the git client you're using.



When you have a merge conflict you'll see all the incoming changes in your stage. Some of them are marked as conflicted. **Don't touch the other files.** To resolve files you'll probably have to make a decision on which version to keep (most conflicts can't be easily manually resolved). In SourceTree you resolve conflicts by right-clicking the file and selecting "Resolve Conflict"

then choose “Mine” (local copy) or “Theirs” (incoming remote change). Once all merge conflicts are resolved you need to mark the merge conflict as complete and continue the merge.





In some git clients there will be an option to continue the merge. For some reason SourceTree doesn’t have this button unless you’re doing a rebase. For any git client you can complete the merge by opening the terminal and typing

```
git merge --continue --no-edit
```

> You may see conflicts on protolibrary.bytes or I2Languages.asset. These are generated files which get changed by a lot of people. Just take “theirs” and don’t worry about it. You or the next person can generate them afresh.

Rebase vs Merge

Git has two options when pulling changes on top of your local commits. A **Merge** is the base case where git will try to apply both sets of changes together and complain if there are merge conflicts. Your git client may show many more commits than you’ve made as being “ready to

push." These are the incoming commits that you've successfully integrated with your changes. A merge is used to bring changes from another branch into the main branch.

A **Rebase** is a good option if you're working directly in the branch you're making changes in (not a side branch). It will basically stash your changes, update your local repo to be in sync with the remote, then apply your changes on top of the new "base." This keeps the git history a little cleaner because it doesn't have to resolve each commit one by one. You ***cannot*** use rebase if you're working in a side branch because this action rewrites git history (bad) so merge is usually a safer option except for simple cases.

Remember if you pull *before* making your commit you can avoid some of the more annoying steps of merge conflict resolution.

FAQs

I tried pulling and got an error!

You probably have local changes that haven't been committed yet. The changes you're fetching from remote may conflict with one or more of your files. Check the error log for the names of any files being modified and discard your local change. (You can redo the change after pulling if you need to)

I tried pushing and got an error!

You could have an unresolved merge conflict. You could have committed a locked file (such as GvhSettings) by accident. You might just be out of date and need to pull again. Ask a dev if none of these fixes work for you.

I got a merge conflict! What do I do?

See the Merge Conflicts section above

I pulled and now see a bunch of commits I didn't make!

When you merge instead of rebase, git will create a Merge Commit. This is where the history of your merge is tracked so git knows any merge conflicts were resolved. Your git client will show all the commits in between when you last pulled (or branched) and now to track the history. Don't panic this is intended.

Oops I committed something I didn't mean to / I want to change my commit!

First verify you have exactly one unpushed commit you want to undo. Be careful if you've pulled new changes after making your commit, you may have a merge commit alongside your bad commit. Once confirmed, open the terminal and type

```
git reset HEAD~1
```

This will “pop” your commit and restage all your changes. You can now discard/modify/whatever freely and commit again once ready.

I don’t know what happened! Help!

Stop clicking buttons, stop doing anything. Reach out to an engineer for help. The more things you change in your git workspace the harder it’ll be to get you back to a good state.

Everything is on fire! I want to start over!

The nuclear option is to do a hard reset. This will destroy all of your local changes and commits and put you back in sync with the remote repository. Open the terminal and type

```
git reset --hard origin/<YOUR_CURRENT_WORKING_BRANCH_NAME>
```

This is the nuclear option. You should very rarely need to use this.

I pulled, opened Unity, and got a bunch of error messages!

Copy the first error message and report the issue to an engineer. Some code or validation issue is breaking the game

My Git client has a Revert/Reverse button. What does that do?

Don’t touch that. Revert is *not* the same as Undo. It will create a new commit that is basically the inverse of the commit, which will confuse the Git history. Use git reset instead

What are these .meta / addressable group files that keep appearing in my stage?

Unity will automatically generate or modify some files to use internally. A .meta file indicates either a directory or a file’s metadata was not submitted, or a directory was emptied but not deleted.

For .meta files run **SquidGames/Tools/Delete Empty Directories**, and see if that removes the rogue meta file. If not it’s possible someone added a new file and forgot to push the meta with it. In that case include the meta.

For addressable groups, these are also generated when new assets are added to the project or moved. Run **SquidGames/Addressables/Update and Clean Groups** to get addressable groups up to date.